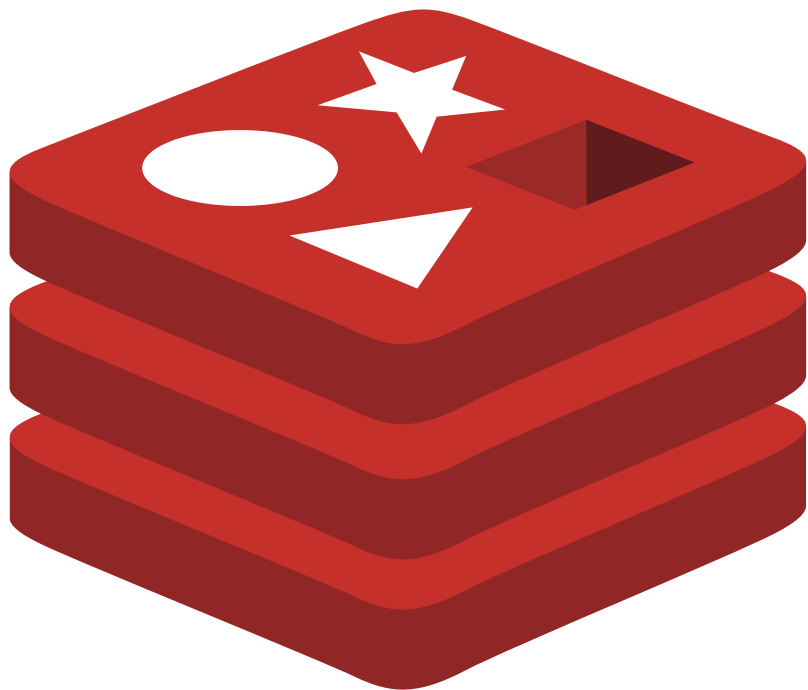


Redis (Redis Server)



Redis — це NoSQL база даних типу ключ-значення, що працює в оперативній пам'яті (RAM).

Вона дозволяє зберігати дані у вигляді хеш-мап без потреби в схемі бази даних.

Дані зберігаються в оперативній пам'яті, що забезпечує надзвичайно швидкий доступ без затримок, характерних для жорстких дисків або SSD.

Клієнтські бібліотеки доступні для більшості популярних мов, включаючи Perl, Python, PHP, Java, Ruby.

Redis ідеально підходить для:

Кешування: Швидкий доступ до часто використовуваних даних.

Управління сесіями: Зберігання інформації про активні сесії користувачів.

Лічильники та конфігурації: Зберігання простих даних, які потребують швидкого доступу.

Основні характеристики Redis

Висока продуктивність:

Операції читання і запису виконуються майже миттєво завдяки зберіганню даних в оперативній пам'яті. Підтримка до мільйонів запитів в секунду.

Різноманітні типи даних:

Strings, Lists, Sets, Sorted Sets, Hashes, Streams, Geospatial Indexes, Bitmaps, Bitfields.

Атомарні операції:

Команди Redis є атомарними, що означає, що вони завжди або успішно виконуються, або зазнають невдачі, без часткового виконання.

Підтримка Pub/Sub:

Redis може функціонувати як шина повідомлень, дозволяючи системам спілкуватися асинхронно.

Реплікація і кластеризація:

Підтримка реплікації для підвищення надійності та доступності.
Кластеризація для горизонтального масштабування.

Збереження даних:

Підтримка збереження даних на диск для забезпечення стійкості до збоїв.

Переваги та недоліки Redis

Основні переваги Redis:

Швидкість: Миттєвий доступ до даних завдяки зберіганню в оперативній пам'яті.

Простота: Зберігання простих пар ключ-значення, зазвичай у вигляді рядків.

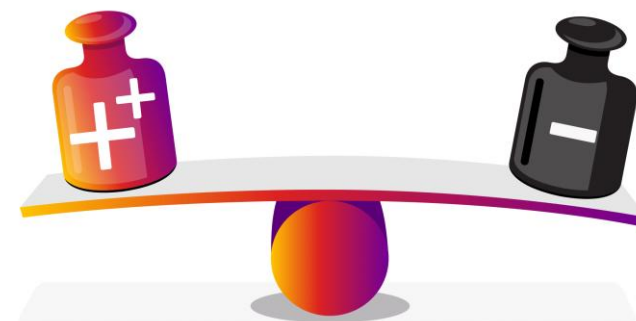
Висока продуктивність, багатий набір функцій, підтримка різних типів даних.

Недоліки:

Волатильність: Дані втрачаються у випадку вимкненні живлення, тому важливі дані потрібно зберігати додатково.

Відсутність схеми: Складно створювати складні моделі даних.

Ціна: Зберігання даних в оперативній пам'яті може бути дорогим для великих обсягів даних.



Переваги та недоліки Redis

Redis рідко використовується як основна база даних, але є чудовим допоміжним інструментом для зберігання швидкодоступної інформації, що покращує загальну продуктивність системи.

The Redis logo, featuring the word "Redis" in a stylized, red, cursive font.

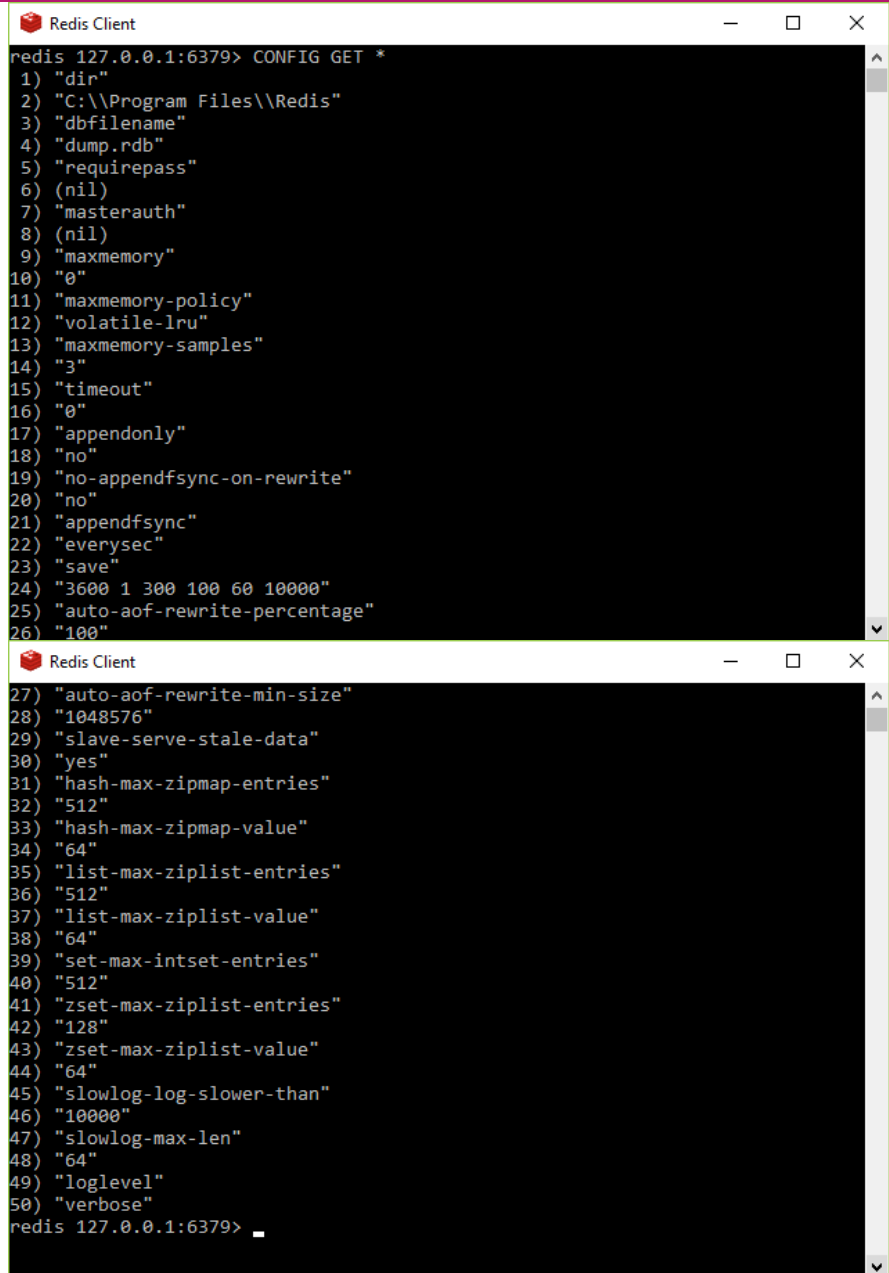
Конфігурації Redis

Конфігураційний файл Redis зазвичай називається **redis.conf**. Він містить різноманітні параметри, які дозволяють налаштувати поведінку сервера Redis.

Файл конфігурації (redis.conf), доступний у кореневому каталозі Redis. Хоча ви можете отримати та встановити всі конфігурації Redis командою Redis **CONFIG**.

Ось основні параметри конфігурації:

- Загальні налаштування
- Налаштування безпеки
- Налаштування збереження даних
- Налаштування реплікації
- Налаштування кластеризації
- Налаштування пам'яті
- Налаштування мережі



```
Redis Client
redis 127.0.0.1:6379> CONFIG GET *
1) "dir"
2) "C:\\Program Files\\Redis"
3) "dbfilename"
4) "dump.rdb"
5) "requirepass"
6) (nil)
7) "masterauth"
8) (nil)
9) "maxmemory"
10) "0"
11) "maxmemory-policy"
12) "volatile-lru"
13) "maxmemory-samples"
14) "3"
15) "timeout"
16) "0"
17) "appendonly"
18) "no"
19) "no-appendfsync-on-rewrite"
20) "no"
21) "appendfsync"
22) "everysec"
23) "save"
24) "3600 1 300 100 60 10000"
25) "auto-aof-rewrite-percentage"
26) "100"

Redis Client
27) "auto-aof-rewrite-min-size"
28) "1048576"
29) "slave-serve-stale-data"
30) "yes"
31) "hash-max-ziplist-entries"
32) "512"
33) "hash-max-ziplist-value"
34) "64"
35) "list-max-ziplist-entries"
36) "512"
37) "list-max-ziplist-value"
38) "64"
39) "set-max-intset-entries"
40) "512"
41) "zset-max-ziplist-entries"
42) "128"
43) "zset-max-ziplist-value"
44) "64"
45) "slowlog-log-slower-than"
46) "10000"
47) "slowlog-max-len"
48) "64"
49) "loglevel"
50) "verbose"
redis 127.0.0.1:6379> _
```

Типи даних Redis

Типи даних Redis

- **Рядки** (Strings)
- **Списки** (Lists)
- **Множини** (Sets)
- **Сортовані множини** (Sorted Sets)
- **Хеші** (Hashes)
- **Потоки** (Streams)
- **Бітові масиви** (Bitmaps)
- **Бітові поля** (Bitfields)
- **Геопросторові індекси** (Geospatial indexes)

Розширені типи даних

- JSON
- Ймовірнісні типи даних (Probabilistic Data Types)
 - HyperLogLog
 - Bloom filter
 - Cuckoo filter
 - t-digest
 - Top-K
 - Count-min sketch
- Часові ряди (Time Series)

Типи даних Redis – Рядки

>_ Redis CLI Python Node.js Java Go C#



```
> SET bike:1 Deimos
OK
> GET bike:1
"Deimos"
```

>_ Redis CLI Python Node.js Java Go C#



```
> mset bike:1 "Deimos" bike:2 "Ares" bike:3 "Vanth"
OK
> mget bike:1 bike:2 bike:3
1) "Deimos"
2) "Ares"
3) "Vanth"
```

Рядки (Strings)

Рядки в Redis є найпростішим типом даних, що представляють собою послідовність байтів.

Рядки в Redis є бінарними. Можна зберігати що-небудь до 512 мегабайт в одному рядку.

В Redis немає числових значень. Тому, якщо хочете зберігати лічильник, Redis має деякі цікаві функції для підтримки цього, але вони завжди будуть рядками.

Типи даних Redis – Списки

>_ Redis CLI Python Node.js Java Go C#

```
> LPUSH bikes:repairs bike:1
(integer) 1
> LPUSH bikes:repairs bike:2
(integer) 2
> RPOP bikes:repairs
"bike:1"
> RPOP bikes:repairs
"bike:2"
```

Списки (Lists)

Списки в Redis — це списки рядків, відсортовані за порядком вставки.

Ви можете додавати значення як з кінця, так і з початку списку.

Списки чудово підходять для колекцій.

Ви можете зберігати до чотирьох мільярдів елементів у списку. Так, це дійсно мільярд. І навіть великі списки мають чудову продуктивність.

Типи даних Redis – Множини

>_ Redis CLI Python Node.js Java Go C#

```
> SADD bikes:racing:france bike:1
(integer) 1
> SADD bikes:racing:france bike:1
(integer) 0
> SADD bikes:racing:france bike:2 bike:3
(integer) 2
> SADD bikes:racing:usa bike:1 bike:4
(integer) 2
```

Множини (Sets)

Множини в Redis — це неупорядковані колекції унікальних рядків.

Ви можете додавати, видаляти та перевіряти наявність елементів за час $O(1)$ (операція виконується за фіксований час, незалежно від розміру вхідних даних).

Це по суті списки, які не дозволяють повторювані значення.

Вони є чудовими інструментами, коли вам потрібно зберігати список значення, і ці значення, можуть повторюватися у вашій логіці. Вам не потрібно перевіряти список на наявність значень, ви можете просто використовувати множину і забути про перевірку.

Типи даних Redis – Сортвані множини

```
> _ Redis CLI Python Node.js Java Go C#

> ZRANGE racer_scores 0 -1
1) "Ford"
2) "Sam-Bodden"
3) "Norem"
4) "Royce"
5) "Castilla"
6) "Prickett"
> ZREVRANGE racer_scores 0 -1
1) "Prickett"
2) "Castilla"
3) "Royce"
4) "Norem"
5) "Sam-Bodden"
6) "Ford"
```

Сортвані множини (Sorted Sets)

Сортвані множини в Redis — це колекції унікальних рядків, які підтримують порядок за асоційованим балом кожного рядка.

Сортвані множини будуються на основі попередніх і є множинами з усіма їхніми властивостями, але також автоматично сортуються за певним значенням.

Ви можете використовувати цей тип даних для зберігання, наприклад, таблиці лідерів у іграх або списку пріоритетів. Сортвання здійснюється за допомогою ефективного алгоритму, тому це чудовий інструмент, коли порядок елементів важливий для вашої бізнес-логіки.

Типи даних Redis – Хеші

> _ Redis CLI

Python

Node.js

Java

Go

C#



```
> HSET bike:1 model Deimos brand Ergonom type 'Enduro bikes' price 4972
(integer) 4
> HGET bike:1 model
"Deimos"
> HGET bike:1 price
"4972"
> HGETALL bike:1
1) "model"
2) "Deimos"
3) "brand"
4) "Ergonom"
5) "type"
6) "Enduro bikes"
7) "price"
8) "4972"
```

Хеші (Hashes)

Хеші в Redis — це типи записів, змодельовані як колекції пар "поле-значення". Це карти між рядковими полями та рядковими значеннями.

Ви можете зберігати мапу пар "ключ-значення", асоційованих з одним ключем. Це зазвичай використовується для представлення більш складних даних, таких як інформація про користувача.

Типи даних Redis – Потoki

> _ Redis CLI Python Node.js Java C#

```
> XADD race:france * rider Castilla speed 30.2 position 1 location_id 1
"1692632086370-0"
> XADD race:france * rider Norem speed 28.8 position 3 location_id 1
"1692632094485-0"
> XADD race:france * rider Prickett speed 29.7 position 2 location_id 1
"1692632102976-0"
```

> _ Redis CLI Python Node.js Java C#

```
> XRANGE race:france 1692632086370-0 + COUNT 2
1) 1) "1692632086370-0"
   2) 1) "rider"
      2) "Castilla"
      3) "speed"
      4) "30.2"
      5) "position"
      6) "1"
      7) "location_id"
      8) "1"
2) 1) "1692632094485-0"
   2) 1) "rider"
      2) "Norem"
      3) "speed"
      4) "28.8"
      5) "position"
      6) "3"
      7) "location_id"
      8) "1"
```

Потоки (Streams)

Потоки в Redis — це структура даних, яка діє як журнал тільки для додавання. Вони допомагають записувати події в порядку їх виникнення та розподіляти їх для обробки.

Це досить нова функція для традиційних баз даних, оскільки зазвичай дані запитуються клієнтами, а не навпаки. За допомогою потоків ви можете налаштувати базу даних на відправку інформації клієнтам, як тільки вона стане доступною.

Типи даних Redis – Інші

Геопросторові індекси (Geospatial Indexes) в Redis корисні для пошуку місць у заданому географічному радіусі або обмеженій області.

Бітові масиви (Bitmaps) в Redis дозволяють виконувати побітові операції над рядками.

Бітові поля (Bitfields) в Redis ефективно кодують кілька лічильників у значенні рядка. Вони забезпечують атомарні операції отримання, встановлення та інкременту і підтримують різні політики переповнення.

Redis **JSON** надає структуровані, ієрархічні масиви та об'єкти ключ-значення, що відповідають популярному формату текстових файлів JSON. Ви можете імпортувати текст JSON у об'єкти Redis і отримувати доступ, змінювати та запитувати окремі елементи даних.

Часові ряди (Time Series) у Redis дозволяють зберігати та запитувати дані з часовими мітками.

Типи даних Redis – Ймовірнісні типи даних

Ймовірнісні типи даних (Probabilistic Data Types) дозволяють збирати та обчислювати статистику приблизно, але дуже ефективно.

HyperLogLog:

Надає ймовірнісні оцінки кардинальності великих множин.

Bloom Filter:

Дозволяє перевіряти наявність або відсутність елемента в множині.

Cuckoo Filter:

Подібний до Bloom Filter, але з дещо іншими компромісами між функціями та продуктивністю.

t-digest:

Оцінює перцентилі з потоку значень даних.

Top-K:

Оцінює рейтинг точки даних у потоці значень.

Count-min Sketch:

Оцінює частоту точки даних у потоці значень.

Команди Redis

Список основних команд Redis

Ключі (Keys)

DEL *key* [*key* ...] – Видаляє один або більше ключів.

EXISTS *key* [*key* ...] – Перевіряє існування одного або більше ключів.

EXPIRE *key seconds* [**NX** | **XX** | **GT** | **LT**] – Встановлює час життя ключа в секундах.

TTL *key* – Повертає залишок часу життя ключа в секундах.

TYPE *key* – Повертає тип збереженого значення за ключем.

Список основних команд Redis

Рядки (Strings)

SET key value [EX seconds] [PX milliseconds] [NX|XX] – Встановлює значення для ключа.

GET key – Отримує значення за ключем.

INCR key – Збільшує числове значення ключа на одиницю.

DECR key – Зменшує числове значення ключа на одиницю.

APPEND key value – Додає значення до кінця існуючого значення ключа.

Список основних команд Redis

Списки (Lists)

LPUSH *key value [value ...]* – Додає один або більше елементів до початку списку.

RPUSh *key value [value ...]* – Додає один або більше елементів до кінця списку.

LPOP *key* – Видаляє і повертає перший елемент списку.

RPOP *key* – Видаляє і повертає останній елемент списку.

LRANGE *key start stop* – Отримує підмножину елементів списку.

Список основних команд Redis

Множини (Sets)

SADD *key member [member ...]* – Додає один або більше елементів до множини.

SREM *key member [member ...]* – Видаляє один або більше елементів з множини.

SMEMBERS *key* – Повертає всі елементи множини.

SISMEMBER *key member* – Перевіряє, чи є елемент членом множини.

SCARD *key* – Повертає кількість елементів у множині.

Список основних команд Redis

Сортовані множини (Sorted Sets)

ZADD key [NX|XX] [CH] [INCR] score member [score member ...] – Додає один або більше елементів до сортованої множини.

ZREM key member [member ...] – Видаляє один або більше елементів з сортованої множини.

ZRANGE key start stop [WITHSCORES] – Повертає елементи з сортованої множини за діапазоном індексів.

ZRANK key member – Повертає ранг (індекс) елемента в сортованій множині.

ZCARD key – Повертає кількість елементів у сортованій множині.

Список основних команд Redis

Хеші (Hashes)

HSET *key field value [field value ...]* – Встановлює поле в хеші.

HGET *key field* – Отримує значення поля в хеші.

HDEL *key field [field ...]* – Видаляє одне або більше полів з хешу.

HGETALL *key* – Повертає всі поля і значення в хеші.

HLEN *key* – Повертає кількість полів у хеші.

Список основних команд Redis

Потоки (Streams)

`XADD key ID field value [field value ...]` – Додає запис до потоку.

`XREAD [COUNT count] [BLOCK milliseconds] STREAMS key [key ...] ID [ID ...]` – Читає записи з потоків.

`XDEL key ID [ID ...]` – Видаляє один або більше записів з потоку.

`XLEN key` – Повертає кількість записів у потоці.

`XRANGE key start end [COUNT count]` – Повертає діапазон записів у потоці.

Список основних команд Redis

Геопросторові індекси (Geospatial Indexes)

`GEOADD key longitude latitude member [longitude latitude member ...]` – Додає геопросторові елементи до індексу.

`GEODIST key member1 member2 [unit]` – Повертає відстань між двома геопросторовими елементами.

`GEORADIUS key longitude latitude radius unit [WITHCOORD] [WITHDIST] [WITHHASH] [COUNT count] [ASC|DESC]` – Повертає елементи в заданому радіусі.

`GEOPOS key member [member ...]` – Повертає координати для одного або більше елементів.

Список основних команд Redis

Бітові масиви (Bitmaps)

SETBIT *key offset value* – Встановлює біту в заданому зсуві.

GETBIT *key offset* – Отримує біту в заданому зсуві.

BITCOUNT *key [start end]* – Повертає кількість встановлених бітів (1) у заданому діапазоні.

Список основних команд Redis

Бітові поля (Bitfields)

`BITFIELD key [GET type offset] [SET type offset value] [INCRBY type offset increment] [OVERFLOW WRAP|SAT|FAIL]` – Виконує атомарні операції над бітовими полями.

Список основних команд Redis

JSON

JSON.SET *key path value* – Встановлює значення JSON за заданим шляхом.

JSON.GET *key [path ...]* – Отримує значення JSON за заданим шляхом.

JSON.DEL *key [path ...]* – Видаляє значення JSON за заданим шляхом.

Список основных команд Redis

Часові ряди (Time Series)

`TS.ADD key timestamp value [RETENTION retention] [LABELS label value ...]` – Додає точку даних до часового ряду.

`TS.RANGE key fromTimestamp toTimestamp [AGGREGATION aggregationType timeBucket]` – Повертає діапазон точок даних.

Список основных команд Redis

Ймовірнісні типи даних (Probabilistic Data Types)

HyperLogLog

PFADD key element [element ...] – Додає елементи до HyperLogLog.

PFCOUNT key [key ...] – Повертає оцінку кардинальності множини.

Bloom Filter

BF.ADD key item – Додає елемент до Bloom Filter.

BF.EXISTS key item – Перевіряє наявність елемента в Bloom Filter.

Cuckoo Filter

CF.ADD key item – Додає елемент до Cuckoo Filter.

CF.EXISTS key item – Перевіряє наявність елемента в Cuckoo Filter.

Список основных команд Redis

Ймовірнісні типи даних (Probabilistic Data Types)

t-digest

`TDIGEST.ADD key value [value ...]` – Додає значення до t-digest.

`TDIGEST.QUANTILE key quantile` – Повертає оцінку перцентилля.

Top-K

`TOPK.ADD key item [item ...]` – Додає елементи до Top-K.

`TOPK.LIST key` – Повертає список Top-K елементів.

Count-min Sketch

`CMS.INCRBY key item increment [item increment ...]` – Збільшує частоту елемента.

`CMS.QUERY key item [item ...]` – Повертає оцінку частоти елемента.

Приклади команди Redis

Рядки (Strings) – SET

Команда **SET** є однією з найчастіше використовуваних у Redis. Вона дозволяє зберігати значення за ключем.

Ось основний синтаксис: **SET ключ значення**. Найважливіші параметри — це ключ і значення.

Ключ можна уявити як контейнер для значення, подібно до змінної в коді. У Redis значення завжди є рядками.

Команда **SET є атомарною**, тобто вона або успішно виконується, або зазнає невдачі, без часткового збереження даних. Якщо ви отримали підтвердження від Redis, дані збережені.

Приклад використання:

SET name "Fernando" — зберігає значення "Fernando" за ключем "name".

SET city "Madrid, Spain" — зберігає значення "Madrid, Spain" за ключем "city".

```
127.0.0.1:6379> SET name Fernando
OK
127.0.0.1:6379> SET city_country "Madrid, Spain"
OK
127.0.0.1:6379> █
```

Якщо ключ не існує, команда SET створить його автоматично.

Вам не потрібно перевіряти наявність ключа перед його використанням.

Команда SET дозволяє зберігати дані в Redis надійно і без зайвих перевірок.

Рядки (Strings) – GET

Команда **GET** є протилежністю до SET. Вона дозволяє отримувати дані, збережені за ключем.

Синтаксис простий: **GET ключ**.

Результати команди GET:

- Рядок, що зберігається за ключем.
- **nil**, якщо ключ порожній або не існує.

Приклад використання:

GET name — повертає значення, збережене за ключем "name".

GET "my first name" — повертає nil, якщо ключ не існує.

Якщо ключ містить пробіли, використовуйте подвійні лапки.

```
127.0.0.1:6379> GET name
"Fernando"
127.0.0.1:6379> GET "my first name"
(nil)
127.0.0.1:6379> SET "my first name" Fernando
OK
127.0.0.1:6379> GET "my first name"
"Fernando"
127.0.0.1:6379> █
```

Підсумок: Команда GET повертає рядок, збережений за ключем, або **nil**, якщо ключ не існує.

Рядки (Strings)

Команди SET і GET є базовими, але не надають багато контролю над рядками. На щастя, Redis пропонує інші команди для роботи з рядками, які є найпоширенішим типом даних у цій базі.

Робота з числами

Redis не працює з числами напряду, але ви можете використовувати рядки як числа за допомогою команд **INCR** і **DECR**, які збільшують або зменшують значення на одиницю.

SET counter 0 — ініціалізує лічильник.

INCR counter — збільшує значення на 1 (результат: 1).

DECR counter — зменшує значення на 1 (результат: 0).

Також є команди **INCRBY** і **DECRBY** для збільшення або зменшення на задане значення.

INCRBY counter 4 — збільшує значення на 4 (результат: 4).

DECRBY counter 2 — зменшує значення на 2 (результат: 2).

```
127.0.0.1:6379> SET counter 0
OK
127.0.0.1:6379> GET counter
"0"
127.0.0.1:6379> incr counter
(integer) 1
127.0.0.1:6379> incr counter
(integer) 2
127.0.0.1:6379> decr counter
(integer) 1
127.0.0.1:6379> incrby counter 4
(integer) 5
127.0.0.1:6379> decrby counter 2
(integer) 3
127.0.0.1:6379> get counter
"3"
127.0.0.1:6379> █
```

Рядки (Strings)

Довжина рядка і об'єднання рядків

STRLEN key — повертає довжину рядка.

APPEND key value — додає значення до кінця існуючого рядка.

SET name "Fernando"

APPEND name "Alonso" — результат: "Fernando Alonso".

Отримання частини рядка

GETRANGE key start end — повертає частину рядка за індексами.

GETRANGE series 0 3 — повертає перші 4 символи рядка.

Підсумок

Використовуйте INCR і DECR для роботи з числовими значеннями.

Використовуйте STRLEN для отримання довжини рядка.

Використовуйте APPEND для об'єднання рядків.

```
127.0.0.1:6379> APPEND series:1662100505 0024
(integer) 4
127.0.0.1:6379> APPEND series:1662100505 0523
(integer) 8
127.0.0.1:6379> APPEND series:1662100505 0612
(integer) 12
127.0.0.1:6379> APPEND series:1662100505 2324
(integer) 16
127.0.0.1:6379> STRLEN series:1662100505
(integer) 16
127.0.0.1:6379> GET series:1662100505
"0024052306122324"
127.0.0.1:6379> GETRANGE series:1662100505 0 3
"0024"
127.0.0.1:6379> █
```

Використання серій (time series key)

Команди **APPEND** і **STRLEN** для створення часових ключів серій з фіксованою довжиною значень.

Додавання значень до серії:

APPEND series "1234"

APPEND series "5678"

APPEND series "9101"

APPEND series "1121"

Отримання довжини серії:

STRLEN series — повертає 16 (довжина всіх значень).

Довжина серії (16) / фіксована довжина значення (4) = 4 значення.

Отримання окремих значень:

GETRANGE series 0 3 — повертає "1234".

GETRANGE series 4 7 — повертає "5678".

Хеші (Hashes)

HashMap — це поширена структура даних у розробці програмного забезпечення, яка використовується і в Redis. Вона представляє собою мапу, де кожен ключ відповідає одному значенню.

Redis можна уявити як велику HashMap у пам'яті, з можливістю зберігати кілька властивостей в одній мапі, що ідеально підходить для складних структур, таких як об'єкти.

Якщо вам потрібно зберегти користувача, кошик для покупок або будь-яку сутність з кількома властивостями, HashMap — це те, що вам потрібно.

Redis оптимізував HashMap до такої міри, що вони можуть зберігати до чотирьох мільярдів властивостей в одній мапі. Це робить їх універсальними для різних випадків використання

Хеші (Hashes)

Взаємодія з хеш-мапами майже така ж проста, як і з простими ключами. Але замість використання команд GET і SET, треба використати **HGET** і **HSET**.

Уявімо, що ми хочемо зберегти інформацію про авторизованого користувача в пам'яті для легкого доступу. Ми не переймаємося втратою цієї версії даних, оскільки вони зберігаються десь ще і можуть бути швидко відновлені.

Дані для зберігання

- Ідентифікатор користувача (user ID)
- Ім'я користувача (name)
- URL аватара користувача (avatarURL)
- Роль користувача (role)

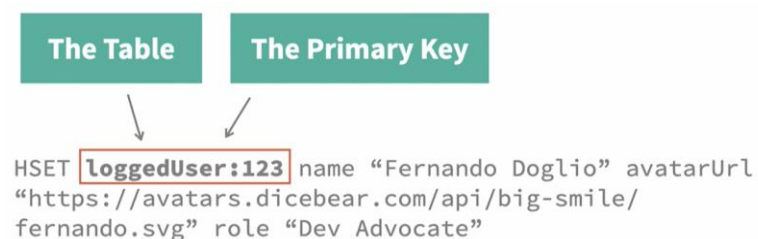
Збереження даних

1. Використовуємо команду HSET.
2. Ключ хеш-мапи: loggedUser:123 (де 123 — це id користувача).
3. Додаємо атрибути та їх значення

Отримання даних

- Використовуємо команду HGET для отримання окремих властивостей.
- Ключ: loggedUser:123
- Атрибут: name

```
-- redis-server 127.0.0.1:6379 ptr_munge=
-- redis-cli
127.0.0.1:6379> HSET loggedUser:123 name "Fernando Doglio" avatarUrl "https://avatars.dicebear.com/api/big-smile/fernando.svg" role "Dev Advocate"
(integer) 3
127.0.0.1:6379> HGET loggedUser:123 name
"Fernando Doglio"
127.0.0.1:6379> HMGET loggedUser:123 name avatarUrl
1) "Fernando Doglio"
2) "https://avatars.dicebear.com/api/big-smile/fernando.svg"
127.0.0.1:6379>
```



To Read More Than One Key

HMGET [hashmap-key] [key 1] [key 2] ... [key N]

Хеші (Hashes)

Припустимо, ми зберігаємо поточний кошик покупок наших користувачів у Redis.
Це чудовий випадок використання, оскільки ми маємо справу з даними, які є тимчасовими.

Використовуємо команду **HSET** для створення нового кошика і додавання даних до нього.

Використовуємо команду **HINCRBY** для збільшення значення (оновлення даних).

Використовуємо команду **HGETALL** для отримання всіх продуктів і їх властивостей.

Використовуємо команду **HLEN** для підрахунку кількості полів у хеш-мапі.

```
~ -- redis-server 127.0.0.1:6379 ptr_munge=
127.0.0.1:6379> HSET cart:123 prod:1:name "Oranges" prod:1:price "2,23" prod:1:amount "1" prod:2:name "Apples" prod:2:price "3,40" prod:2:amount 4
(integer) 6
127.0.0.1:6379> HINCRBY cart:123 prod:1:amount 3
(integer) 4
127.0.0.1:6379> HGETALL cart:123
1) "prod:1:name"
2) "Oranges"
3) "prod:1:price"
4) "2,23"
5) "prod:1:amount"
6) "4"
7) "prod:2:name"
8) "Apples"
9) "prod:2:price"
10) "3,40"
11) "prod:2:amount"
12) "4"
127.0.0.1:6379> HLEN cart:123
(integer) 6
127.0.0.1:6379> █
```

Key + ID



HSET **cart:123** prod:1:name "Oranges" prod:1:price "2,23" prod:1:amount "1" prod:2:name **"Apples"** prod:2:price "3,40" prod:2:amount "4"



Key + ID + Property

Хеш-мапи дозволяють представляти складні об'єкти в Redis, а не просто ключі та значення.

Списки (Lists)

Списки — це один з тих типів даних, які, як тільки ви про них дізнаєтеся, ви завжди будете намагатися використовувати.

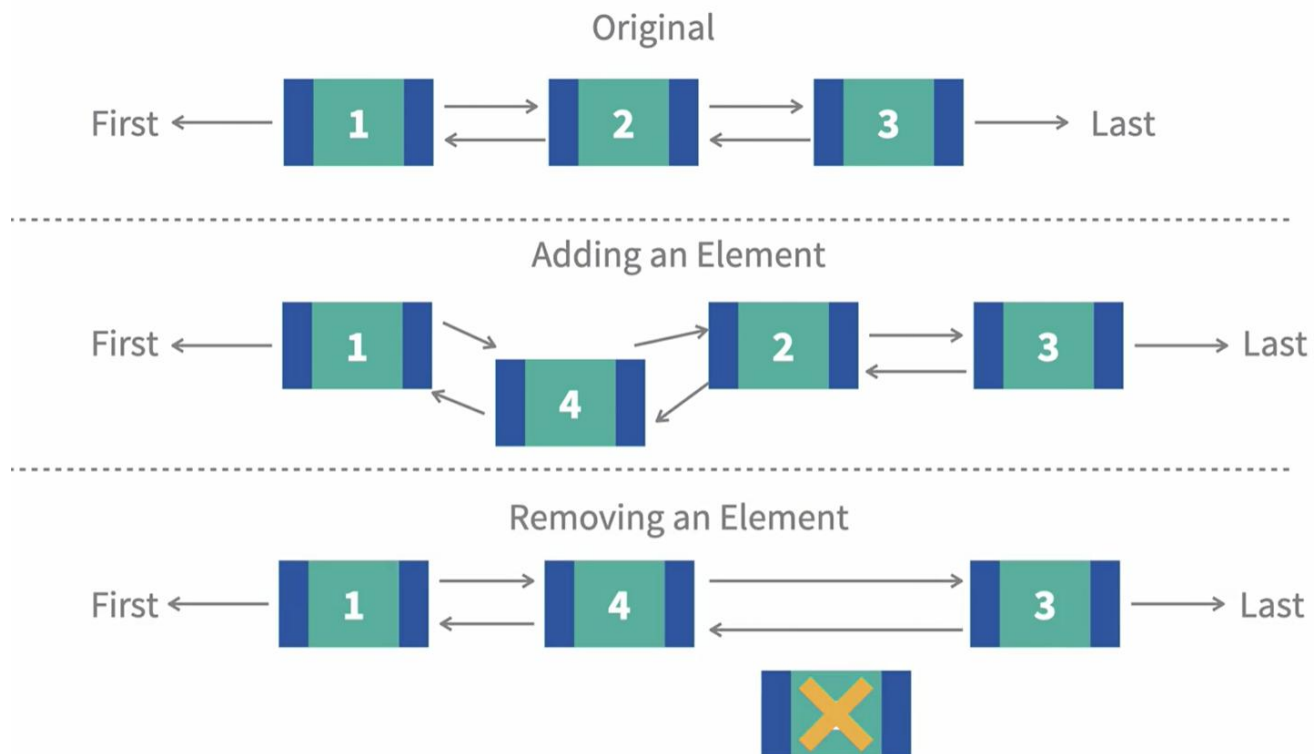
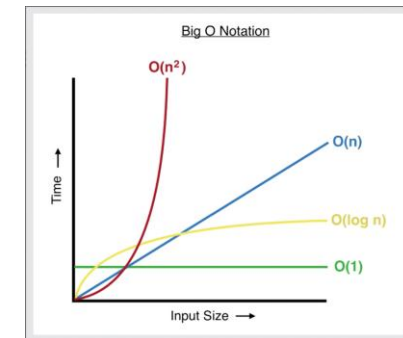
Списки — це просто колекція рядків. Завдяки своїй внутрішній структурі, вони дуже ефективні під час додавання та видалення даних з обох кінців.

Списки реалізовані за допомогою структури даних "двозв'язний список". Це означає, що кожен елемент у списку "знає" про попередній і наступний елемент.

Також це означає, що список має пряме посилання як на перший, так і на останній вузол у списку.

Вони прості у використанні та ефективні для багатьох випадків застосування. Однак, якщо вам потрібно часто взаємодіяти з елементами в середині списку, будьте готові до складності $O(N)$.

Списки дозволяють додавати дублікати, і немає механізму, який би запобігав цьому.



Списки (Lists)

Механізм Pop і Push часто використовується зі списками та типами даних на основі списків.

Команди для роботи зі списками

LPUSH: Додавання елемента до голови списку.

RPUSH: Додавання елемента до хвоста списку.

LPOP: Витягування елемента з голови списку.

RPOP: Витягування елемента з хвоста списку.

```
127.0.0.1:6379> lpush MYLIST "first element" "second element" "final element"
(integer) 3
127.0.0.1:6379> LPOP MYLIST
"final element"
127.0.0.1:6379> █
```

Важливі зауваження

Витягування елемента (pop) змінює список, зменшуючи кількість елементів у ньому.
Додавання елемента (push) збільшує кількість елементів у списку.

LPUSH MYLIST "first element" "second element" "final element"

LPUSH: Додавання елементів до голови списку змінює порядок так, що останній доданий елемент стає першим.

RPUSH MYLIST "first element" "second element" "final element"

RPUSH: Додавання елементів до хвоста списку зберігає порядок вставки.

LPOP MYLIST

LPOP: Витягування одного елемента з голови списку.

LPOP MYLIST 2

LPOP з кількістю: Витягування кількох елементів з голови списку.

Списки (Lists). Практичне використання

Практичне використання списків: Система черг

Ми створимо систему черг, яка дозволить виконувати наступні дії:

1. Додавати людей до черги.
2. Витягувати людей з черги.
3. Перевіряти, хто стоїть у черзі.
4. Дозволяти преміум-клієнтам проходити без черги.
5. Перевіряти, хто стоїть останнім у черзі.

Ми будемо використовувати черги типу:

FIFO (First In, First Out): перший зайшов — перший вийшов,

LIFO (Last In, First Out): останній зайшов — перший вийшов.

Команди для роботи з чергою **FIFO**

RPUSH: Додавання людини до кінця черги.

LPOP: Витягування людини з початку черги.

LRANGE: Перевірка, хто стоїть у черзі.

RPOP: Перевірка, хто стоїть останнім у черзі.

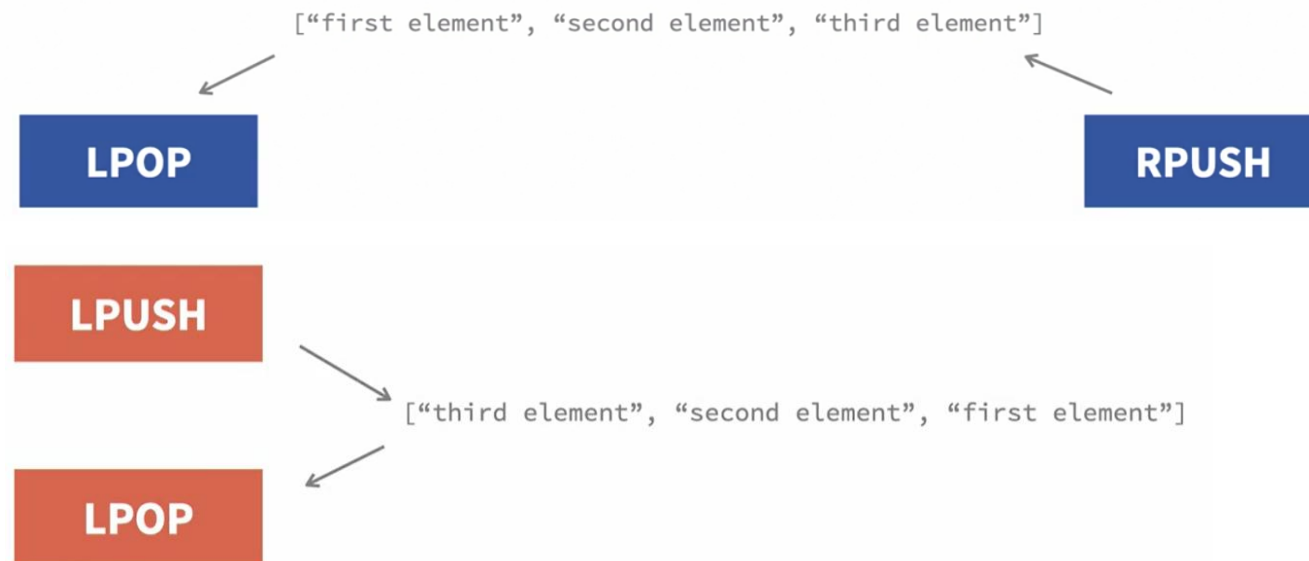
Команди для роботи з чергою **LIFO**

LPUSH: Додавання людини до початку черги.

LPOP: Витягування людини з початку черги.

LRANGE: Перевірка, хто стоїть у черзі.

RPOP: Перевірка, хто стоїть останнім у черзі.



Списки (Lists). Практичне використання

Ми будемо додавати імена людей до нашого списку (черги).

Припустимо, що 3 людини вже стоять біля дверей на момент відкриття, а 2 інших приходять по одному пізніше.

Ми будемо використовувати команду **RPUSH** для додавання імен до списку.

```
127.0.0.1:6379> RPUSH thequeue "Carol Shaw" "Elizabeth Carr" "Ernest Ramos"  
(integer) 3  
127.0.0.1:6379> RPUSH thequeue "Jane Carter"  
(integer) 4  
127.0.0.1:6379> RPUSH thequeue "Martha Cooper"  
(integer) 5  
127.0.0.1:6379> █
```

```
127.0.0.1:6379> LPOP thequeue  
"Carol Shaw"  
127.0.0.1:6379> LPOP thequeue  
"Elizabeth Carr"  
127.0.0.1:6379> LPOP thequeue  
"Ernest Ramos"  
127.0.0.1:6379> LPOP thequeue  
"Jane Carter"  
127.0.0.1:6379> LPOP thequeue  
"Martha Cooper"  
127.0.0.1:6379> LPOP thequeue  
(nil)  
127.0.0.1:6379> █
```

Ми будемо обслуговувати клієнтів по одному, використовуючи команду **LPOP**.

Кожного разу, коли ми використовуємо цю команду, ми отримуємо одне ім'я з початку черги.

Результат: nil (черга порожня)

Списки (Lists). Практичне використання

Інспекція черги без її зміни

Щоб перевірити, хто стоїть у черзі, не витягуючи елементи з неї, ми можемо використовувати команду **LRANGE**. Ця команда дозволяє повернути елементи в межах певного діапазону, не змінюючи сам список.

LRANGE thequeue 0 2:

Повертає елементи з індексами від 0 до 2 (включно).

Індекси починаються з нуля, тому 0 — це перший елемент, а 2 — третій елемент.

LRANGE thequeue 0 -1:

Повертає елементи з індексами від 0 до -1.

Негативні значення починаються з кінця списку, тому -1 — це останній елемент.

Ця команда повертає всі елементи черги, незалежно від їх кількості.

```
127.0.0.1:6379> LRANGE thequeue 0 2
1) "Carol Shaw"
2) "Elizabeth Carr"
3) "Ernest Ramos"
127.0.0.1:6379> LRANGE thequeue 0 -1
1) "Carol Shaw"
2) "Elizabeth Carr"
3) "Ernest Ramos"
4) "Jane Carter"
5) "Martha Cooper"
127.0.0.1:6379> █
```

Списки (Lists). Практичне використання

Впровадження системи "преміум-клієнтів"

Ми хочемо додати преміум-клієнта до середини черги.

Наприклад, черга зараз виглядає так: Carol, Elizabeth, Ernest, Jane, Martha.

Ми хочемо додати преміум-клієнта Patricia Fox після Ernest.

INSERT thequeue AFTER "Ernest Ramos" "Patricia Fox":

Додає **"Patricia Fox"** після **"Ernest Ramos"**.

Команда **INSERT** працює з значеннями, а не з індексами, і дозволяє вставляти новий елемент перед або після вказаного значення використовуючи **BEFORE** та **AFTER**.

INSERT додає нове значення перед або після першої зустрічної інстанції вказаного значення.

Якщо значення повторюється в списку, нове значення буде додано тільки біля першої інстанції цього значення.

```
127.0.0.1:6379> LRANGE thequeue 0 -1
1) "Carol Shaw"
2) "Elizabeth Carr"
3) "Ernest Ramos"
4) "Jane Carter"
5) "Martha Cooper"
127.0.0.1:6379> INSERT thequeue AFTER "Ernest Ramos" "Patricia Fox"
(integer) 6
127.0.0.1:6379> LRANGE thequeue 0 -1
1) "Carol Shaw"
2) "Elizabeth Carr"
3) "Ernest Ramos"
4) "Patricia Fox"
5) "Jane Carter"
6) "Martha Cooper"
127.0.0.1:6379> █
```

Списки (Lists). Практичне використання

Отримання першого та останнього елементів черги без їх видалення

Ми можемо використовувати команду **LRANGE**, щоб отримати перший і останній елементи черги без їх видалення. Для цього потрібно гратися з діапазонами індексів.

LRANGE thequeue 0 0

Черга: ["Carol Shaw", "Elizabeth Carr", "Ernest Ramos", "Patricia Fox", "Jane Carter", "Martha Cooper"]

LLEN thequeue

Результат: 6 (довжина списку)

LRANGE thequeue 5 5

Повертає шостий елемент черги

LRANGE thequeue -1 -1

Повертає останній елемент черги

LRANGE thequeue 0 0

Результат: ["Carol Shaw"]

```
127.0.0.1:6379> LRANGE thequeue 0 -1
1) "Carol Shaw"
2) "Elizabeth Carr"
3) "Ernest Ramos"
4) "Jane Carter"
5) "Martha Cooper"
127.0.0.1:6379> LINSERT thequeue AFTER "Ernest Ramos" "Patricia Fox"
(integer) 6
127.0.0.1:6379> LRANGE thequeue 0 -1
1) "Carol Shaw"
2) "Elizabeth Carr"
3) "Ernest Ramos"
4) "Patricia Fox"
5) "Jane Carter"
6) "Martha Cooper"
127.0.0.1:6379> LRANGE thequeue -1 -1
1) "Martha Cooper"
127.0.0.1:6379>
```

Множини (Sets)

Система черг працює, але має обмеження у вигляді можливості додавання дублікатів.

Це не є проблемою бізнес-логіки, а особливістю роботи списків.

Ви можете контролювати додавання та витягування даних за допомогою механізмів pop і push, інспектувати дані без зміни списку за допомогою команди LRANGE та додавати дублікати, якщо це необхідно.

Для уникнення дублікатів потрібно самостійно перевіряти наявність елементів у списку перед їх додаванням.

Множини (Sets)

Множини (Sets) у Redis дозволяють зберігати групу елементів без дублікатів.

Це робить їх ідеальними для випадків, коли важливо уникнути повторень, як у нашій системі черг. Якщо ви спробуєте вставити той самий елемент більше одного разу, тільки перша спроба буде успішною, а наступні будуть проігноровані.

Множини мають деякі відмінності від списків, зокрема:

- Додавання елементів тільки в кінець множини за допомогою команди SADD.
- Команда SPOP витягує випадкові елементи з множини.
- Команда SREM видаляє конкретні елементи з множини.
- Елементи в множині не мають порядку.

Множини (Sets)

Основні команди для роботи з множинами

SADD: Додавання елемента до множини.

SPOP: Витягування випадкового елемента з множини.

SREM: Видалення конкретного елемента з множини.

SMEMBERS: Отримання всіх елементів множини.

```
127.0.0.1:6379> SADD thequeue "Carol Shaw" "Elizabeth Carr" "Ernest Ramos"  
(integer) 3  
127.0.0.1:6379> SADD thequeue "Jane Carter"  
(integer) 1  
127.0.0.1:6379> SADD thequeue "Martha Cooper"  
(integer) 1  
127.0.0.1:6379> █
```

```
SMEMBERS myqueue  
1) Carol Shaw  
2) Elizabeth Carr  
3) Ernest Ramos  
4) Jane Carter  
5) Martha Cooper
```

```
127.0.0.1:6379> SREM thequeue "Carol Shaw"  
(integer) 1  
127.0.0.1:6379> SMEMBERS thequeue  
1) "Jane Carter"  
2) "Ernest Ramos"  
3) "Martha Cooper"  
4) "Elizabeth Carr"  
127.0.0.1:6379> █
```

Сортовані множини (Sorted Sets)

Впорядковані множини (Sorted Sets) у Redis схожі на звичайні множини тим, що не дозволяють дублікати. Однак вони мають одну важливу особливість — порядок

Елементи у впорядкованій множині мають попередньо визначений порядок, який задається значенням (score), що присвоюється при вставці елемента.

Це дозволяє реалізовувати системи з пріоритетами, лідерборди та системи планування завдань.

Основні команди для роботи з впорядкованими множинами

ZADD: Додавання елемента з певним значенням (score).

ZRANGE: Отримання елементів у заданому діапазоні індексів.

ZREM: Видалення елемента з множини.

ZSCORE: Отримання значення (score) елемента.

ZINCRBY: Збільшення значення (score) елемента.

Сортовані множини (Sorted Sets)

```
127.0.0.1:6379> ZADD leaderboard 304 "Doris Ramirez"
(integer) 1
127.0.0.1:6379> ZADD leaderboard 1000 "Sandra Ortiz"
(integer) 1
127.0.0.1:6379> ZADD leaderboard 860 "Nicholas Clark"
(integer) 1
127.0.0.1:6379> ZADD leaderboard 600 "Cheryl Robinson"
(integer) 1
127.0.0.1:6379> ZADD leaderboard 100 "Brian Mcdonald"
(integer) 1
```

```
127.0.0.1:6379> ZREVRANGE leaderboard 0 5
1) "Sandra Ortiz"
2) "Nicholas Clark"
3) "Cheryl Robinson"
4) "Doris Ramirez"
5) "Brian Mcdonald"
127.0.0.1:6379> ZINCRBY leaderboard 300 "Nicholas Clark"
"1160"
127.0.0.1:6379> ZREVRANGE leaderboard 0 5
1) "Nicholas Clark"
2) "Sandra Ortiz"
3) "Cheryl Robinson"
4) "Doris Ramirez"
5) "Brian Mcdonald"
```

ZADD: Додавання елемента з певним значенням (score).

ZRANGE: Отримання елементів у заданому діапазоні індексів у порядку зростання значень.

ZREVRANGE: Отримання елементів у заданому діапазоні індексів у порядку спадання значень.

ZINCRBY: Збільшення значення (score) елемента.

ZRANGE leaderboard 0 5 **WITHSCORES**: Повертає всіх гравців з їх значеннями

Впорядковані множини (Sorted Sets) у Redis дозволяють легко реалізувати лідерборд, де гравці відсортовані за кількістю очок.

Використовуючи команди ZADD, ZRANGE, ZREVRANGE та ZINCRBY, ви можете ефективно керувати лідербордом, автоматично сортувати гравців за їх значеннями та оновлювати їх бали.

Це значно спрощує роботу з лідербордами та іншими системами, де важливе сортування за значенням.

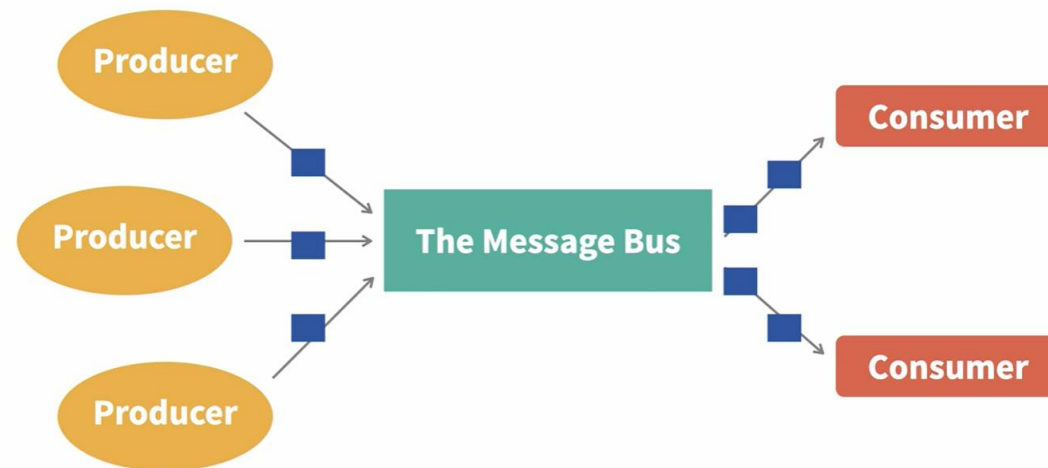
**Що ще може робити
Redis?**

Шина повідомлень (message bus)

Redis — це не просто база даних. Однією з найцікавіших функцій Redis є його здатність працювати як шина повідомлень.

Шина повідомлень — це інфраструктура обміну повідомленнями, яка дозволяє кільком системам спілкуватися одна з одною асинхронно.

Вона розташована між виробником (створює дані) і споживачем (читає дані).



1.

Persist Data

Data is persisted inside the bus.

This is to prevent data loss if there are no consumers available.

Hint: Streams work like this.

2.

Fire and Forget

Data is published whether there are consumers or not.

If there aren't, then the message is lost.

Hint: Pub/Sub works like this.

Існують два основні шаблони:

- дані зберігаються в шині деякий час, якщо немає споживачів, зацікавлених у них.
- дані розподіляються шиною, і якщо на іншому кінці немає споживача, то вони просто втрачаються.

Redis Публікація/Підписка

Pub/Sub — це функція Redis, яка була першою реалізацією шини повідомлень у цій базі даних багато років тому.

Вона дуже проста у використанні і не потребує налаштування. Однак, через свою простоту, вона не є надзвичайно надійною.

Механізм роботи:

Fire-and-forget: Виробник даних надсилає дані в шину і забуває про них. Чи досягнуть дані споживача, не має значення.

Споживач: Отримує дані тільки після підключення до шини. Дані, створені до підключення, не будуть доступні.

Це чудове рішення для реального часу, оскільки воно легке і просте у використанні. Однак, якщо споживачі нестабільні, ви можете втратити деякі дані.



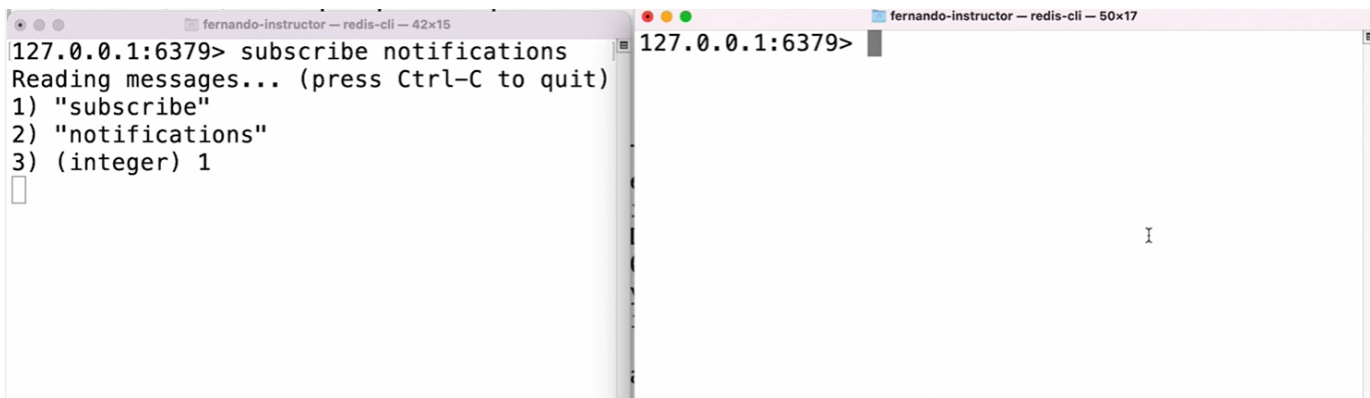
Redis Публікація/Підписка: Приклад використання

Виробник: Використовує команду **PUBLISH** для надсилання повідомлення.

PUBLISH notifications "it's time to learn Redis"

Споживач: Використовує команду **SUBSCRIBE** для підписки на канал.

SUBSCRIBE notifications

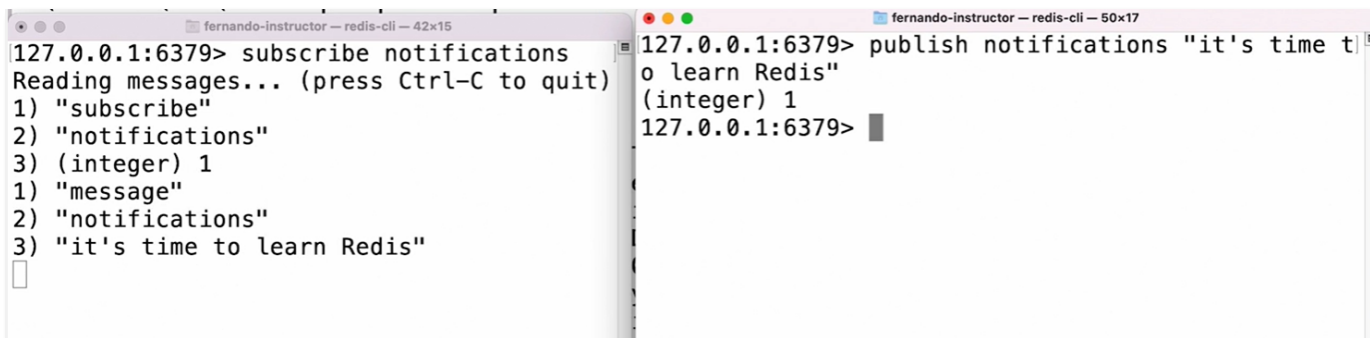


```
fernando-instructor — redis-cli — 42x15
127.0.0.1:6379> subscribe notifications
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "notifications"
3) (integer) 1
└─┘

fernando-instructor — redis-cli — 50x17
127.0.0.1:6379>
```

споживач

виробник



```
fernando-instructor — redis-cli — 42x15
127.0.0.1:6379> subscribe notifications
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "notifications"
3) (integer) 1
1) "message"
2) "notifications"
3) "it's time to learn Redis"
└─┘

fernando-instructor — redis-cli — 50x17
127.0.0.1:6379> publish notifications "it's time to learn Redis"
(integer) 1
127.0.0.1:6379>
```

Демонстрація:

1. Відкрийте два термінали.
2. У першому терміналі (споживач) виконайте **SUBSCRIBE notifications**.
3. У другому терміналі (виробник) виконайте **PUBLISH notifications "it's time to learn Redis"**.
4. Споживач негайно отримає повідомлення.

Таким чином, Pub/Sub у Redis дозволяє легко налаштувати обмін повідомленнями в реальному часі між виробниками і споживачами.

Потоки: Додавання даних

Потоки — це потужний і універсальний тип даних у Redis, схожий на журнали (log files).

Вони не є окремою функцією, як Pub/Sub.

Вони дозволяють додавати дані тільки в кінець і запитувати їх різними способами.

Потоки зберігають дані до тих пір, доки ви спеціально не видалите запис.

Потоки: Додавання даних

Для додавання даних до потоку використовуйте команду `XADD <ключ> <ID> [<ключ> <значення>...]`.

Ви вказуєте **ключ**, **ID** і **набір пар "ключ-значення"**.

```
fernando-instructor — redis-cli — 94x29
127.0.0.1:6379> XADD myevents * user "fernando" event_type "left-click" coord_x 32 coord_y 56
"1662801891882-0"
127.0.0.1:6379> XADD myevents * user "fernando" event_type "right-click" coord_x 53 coord_y 10
0
"1662801936349-0"
127.0.0.1:6379> XADD myevents * user "fernando" event_type "scroll"
"1662801949435-0"
127.0.0.1:6379> █
```

Ви побачите, що повертається часова мітка і автоінкрементне число (0, якщо ви єдиний, хто вводить ці події одночасно).

У прикладі з подіями координати *x* і *y* представляють позицію на екрані, де сталася подія (наприклад, клік миші).

Ключ — це ім'я ключа, де ви будете зберігати потік.

ID — це ідентифікатор даних, які ви додаєте до потоку. Зазвичай ви використовуєте зірочку (*), що означає, що Redis автоматично згенерує цей ID для вас. ID буде унікальною часовою міткою, за якою слідує автоінкрементне число, щоб уникнути колізій.

Пари "ключ-значення" дозволяють вводити складні структури даних у потік. Це не просто рядки. Уявіть, що ви додаєте рядки CSV-файлу. Кожен ключ буде назвою стовпця, а значення — фактичним значенням, яке ви вводите.

Потоки: Читання даних

Читання даних з потоку в Redis може здійснюватися різними способами, залежно від вашого випадку використання.

Читання за діапазоном дат

Оскільки ID елементів у потоці є Unix-мітками часу, ви можете використовувати команду **XRANGE** для вказання початкового і кінцевого часу діапазону.

Синтаксис: **XRANGE** ключ початок кінець [COUNT ліміт]

XRANGE myevents - + — повертає всі дані в потоці.

Мінус (-) означає найменшу можливу мітку часу, а плюс (+) – найбільшу.

XRANGE myevents - + COUNT 10 — повертає перші 10 елементів.

XREVRANGE myevents + - COUNT 1 — повертає останній елемент у потоці.

```
127.0.0.1:6379> XRANGE myevents - +
1) 1) "1662801891882-0"
   2) 1) "user"
      2) "fernando"
      3) "event_type"
      4) "left-click"
      5) "coord_x"
      6) "32"
      7) "coord_y"
      8) "56"
2) 1) "1662801936349-0"
   2) 1) "user"
      2) "fernando"
      3) "event_type"
      4) "right-click"
      5) "coord_x"
      6) "53"
      7) "coord_y"
      8) "100"
3) 1) "1662801949435-0"
   2) 1) "user"
      2) "fernando"
      3) "event_type"
      4) "scroll"
127.0.0.1:6379>
```

```
127.0.0.1:6379> XREVRANGE myevents + - COUNT 1
1) 1) "1662801949435-0"
   2) 1) "user"
      2) "fernando"
      3) "event_type"
      4) "scroll"
127.0.0.1:6379>
```

Потоки: Читання даних

Читання даних у реальному часі

Ви можете підписатися на потік і отримувати сповіщення про нові дані за допомогою команди XREAD.

І це схоже на роботу Pub/Sub у тому сенсі, що кожен доданий елемент буде транслюватися кожному підключеному споживачу. Однак головна відмінність полягає в тому, що ці **записи не будуть видалені з потоку**.

Синтаксис: **XREAD [COUNT число] [BLOCK мс] STREAMS ключ ID**

COUNT число (необов'язковий): Обмежує кількість елементів, які повертаються.

BLOCK мс (необов'язковий): Час очікування в мілісекундах для нових даних.

STREAMS ключ ID: Вказує потік (ключ) і останній прочитаний ID.

1. Відкрийте два термінали.
2. У першому терміналі (споживач) виконайте **XREAD BLOCK 0 STREAMS myevents \$**.

3. У другому терміналі (виробник) виконайте **XADD myevents * user Fernando event_type scroll**.
4. Споживач негайно отримає повідомлення з потоку.

```
fernando-instructor — redis-cli — 93x13
127.0.0.1:6379> XREAD BLOCK 0 STREAMS myevents $

```

```
fernando-instructor — redis-cli — 93x13
127.0.0.1:6379> XREAD BLOCK 0 STREAMS myevents $

```

виробник

```
fernando-instructor — redis-cli — 93x13
127.0.0.1:6379> XREAD BLOCK 0 STREAMS myevents $
1) 1) "myevents"
   2) 1) 1) "1662804443452-0"
        2) 1) "user"
           2) "fernando"
           3) "event_type"
           4) "scroll"
(64.60s)
127.0.0.1:6379> 
```

```
fernando-instructor — redis-cli — 91x14
127.0.0.1:6379> 
```

```
fernando-instructor — redis-cli — 91x14
127.0.0.1:6379> XADD myevents * user "fernando" event_[[NOMKSTREAM] [MAXLEN|MINID

```

споживач

```
fernando-instructor — redis-cli — 91x14
127.0.0.1:6379> XADD myevents * user "fernando" event_type "scroll"
"1662804443452-0"
127.0.0.1:6379> 
```

Безпека Redis

Безпека Redis включає кілька рівнів захисту, таких як аутентифікація, мережева безпека, шифрування, обмеження команд, моніторинг і ізоляція середовища.



Рекомендації з безпеки

Використовуйте сильні паролі: Забезпечте, щоб паролі були складними і важкими для вгадування.

Обмежте мережевий доступ: Дозволяйте доступ до Redis тільки з довірених IP-адрес.

Використовуйте шифрування: Захищайте дані під час передачі за допомогою TLS/SSL.

Регулярно оновлюйте Redis: Встановлюйте останні оновлення і патчі для забезпечення безпеки.

Моніторинг і аудит: Постійно відстежуйте активність і аналізуйте журнали для виявлення підозрілих дій.

Безпека Redis

Аутентифікація

Парольний захист: Redis підтримує простий механізм аутентифікації за допомогою пароля. Ви можете встановити пароль у конфігураційному файлі (`redis.conf`) за допомогою параметра `requirepass`.

Приклад: `requirepass yourpassword`

Команда AUTH: Після встановлення пароля клієнти повинні використовувати команду AUTH, щоб аутентифікуватися перед виконанням інших команд.

Приклад: `AUTH yourpassword`

Безпека Redis

Мережева безпека

Біндінг до локального інтерфейсу: За замовчуванням Redis слухає всі мережеві інтерфейси. Ви можете обмежити доступ, вказавши конкретні IP-адреси в конфігураційному файлі.

Приклад: `bind 127.0.0.1`

Брандмауер: Використовуйте брандмауер для обмеження доступу до порту Redis (за замовчуванням 6379) тільки з довірених IP-адрес.

Безпека Redis

Шифрування

TLS/SSL: Redis підтримує шифрування TLS/SSL для захисту даних під час передачі. Це забезпечує конфіденційність і цілісність даних між клієнтом і сервером.

Налаштування: Включає налаштування сертифікатів і ключів у конфігураційному файлі ([redis.conf](#)).

[tls-cert-file](#) /path/to/redis.crt

[tls-key-file](#) /path/to/redis.key

[tls-ca-cert-file](#) /path/to/ca.crt

Безпека Redis

Обмеження команд

Доступ до команд: Ви можете обмежити доступ до певних команд або повністю їх відключити для підвищення безпеки.

Відключення команди FLUSHALL:

```
rename-command FLUSHALL ""
```


Безпека Redis

Моніторинг і аудит

Журнали: Використовуйте журнали Redis для моніторингу активності і виявлення підозрілих дій.

Приклад: Налаштування рівня логування в конфігураційному файлі:

`loglevel notice`

`logfile /var/log/redis/redis.log`

Моніторинг: Використовуйте інструменти моніторингу, такі як **Redis Sentinel** або сторонні рішення, для відстеження стану і продуктивності сервера.

Безпека Redis

Ізоляція середовища

Контейнери і віртуальні машини: Розгортайте Redis у контейнерах (наприклад, Docker) або віртуальних машинах для ізоляції від інших сервісів і підвищення безпеки.

Обмеження доступу: Використовуйте механізми контролю доступу на рівні операційної системи для обмеження доступу до файлів і процесів Redis.

Redis benchmark

Redis Benchmark — це інструмент для вимірювання продуктивності Redis-сервера. Він дозволяє оцінити швидкість виконання різних команд і загальну продуктивність сервера під навантаженням.

Інструмент **redis-benchmark** постачається разом з Redis і може бути запущений з командного рядка.

redis-benchmark [опції]

Опції:

- h:** Вказує хост сервера Redis. Приклад: `-h 127.0.0.1`
- p:** Вказує порт сервера Redis. Приклад: `-p 6379`
- c:** Вказує кількість одночасних з'єднань. Приклад: `-c 50`
- n:** Вказує кількість запитів. Приклад: `-n 100000`
- d:** Вказує розмір значення в байтах для команд, що працюють з рядками. Приклад: `-d 100`
- t:** Вказує типи команд для тестування. Приклад: `-t set,get`
- q:** Виводить лише підсумкову статистику. Приклад: `-q`

```
$ redis-benchmark -t set -r 100000 -n 1000000
===== SET =====
 1000000 requests completed in 13.86 seconds
 50 parallel clients
 3 bytes payload
 keep alive: 1

99.76% `<=` 1 milliseconds
99.98% `<=` 2 milliseconds
100.00% `<=` 3 milliseconds
100.00% `<=` 3 milliseconds
72144.87 requests per second

$ redis-cli dbsize
(integer) 99993
```

Транзакції в Redis

Транзакції в Redis дозволяють виконувати кілька команд як одну атомарну операцію. Це означає, що всі команди в транзакції будуть виконані або всі разом, або жодна з них не буде виконана, що забезпечує цілісність даних.

Основні команди для роботи з транзакціями

MULTI: Починає транзакцію.

EXEC: Виконує всі команди, додані до транзакції.

DISCARD: Скасовує всі команди, додані до транзакції.

WATCH: Слідкує за змінами ключів і скасовує транзакцію, якщо будь-який з ключів змінюється до виконання команди EXEC.

UNWATCH: Відміняє слідкування за ключами.

Особливості транзакцій в Redis

Атомарність: Всі команди в транзакції виконуються як одна атомарна операція.

Відсутність ізоляції: Команди в транзакції не ізольовані від інших команд, що виконуються одночасно. Це означає, що інші клієнти можуть бачити часткові результати транзакції до її завершення.

Відсутність відкату: Redis не підтримує відкату транзакцій. Якщо одна з команд в транзакції зазнає невдачі, інші команди все одно будуть виконані.

```
redis 127.0.0.1:6379> MULTI
OK
redis 127.0.0.1:6379> SET tutorial redis
QUEUED
redis 127.0.0.1:6379> GET tutorial
QUEUED
redis 127.0.0.1:6379> INCR visitors
QUEUED
redis 127.0.0.1:6379> EXEC
1) OK
2) "redis"
3) (integer) 1
```

```
redis 127.0.0.1:6379> MULTI
OK
redis 127.0.0.1:6379> EXEC
(empty list or set)
redis 127.0.0.1:6379> MULTI
OK
redis 127.0.0.1:6379> SET javatpoint redis
QUEUED
redis 127.0.0.1:6379> GET javatpoint
QUEUED
redis 127.0.0.1:6379> INCR visitors
QUEUED
redis 127.0.0.1:6379> EXEC
1) OK
2) "redis"
3) (integer) 1
```

Redis pipelining

Redis Pipelining — це техніка, яка дозволяє клієнту надсилати кілька команд до сервера без очікування відповідей на кожну з них. Це значно зменшує затримки, пов'язані з мережею, і підвищує продуктивність, особливо під час виконання великої кількості команд.

Як працює Pipelining?

Замість того, щоб надсилати команду, чекати на відповідь і потім надсилати наступну команду, клієнт надсилає всі команди одразу. Сервер Redis обробляє всі команди і повертає відповіді в порядку їх надходження.

Важливо:

Під час використання pipelining сервер формує чергу відповідей у пам'яті.

Тому, якщо потрібно надіслати багато команд, краще робити це партіями, наприклад, по 10 тисяч команд.

Надішліть 10k команд, прочитайте відповіді, потім надішліть наступні 10k команд.

Це забезпечить майже однакову швидкість, але зменшить використання пам'яті.

Без Pipelining:

```
SET key1 value1
// Чекати на відповідь
SET key2 value2
// Чекати на відповідь
SET key3 value3
// Чекати на відповідь
```

3 Pipelining:

```
// Надіслати всі команди одразу
SET key1 value1
SET key2 value2
SET key3 value3
// Отримати всі відповіді одразу
```

Redis pipelining

Переваги Pipelining

Зменшення затримок:

Зменшує кількість мережевих затримок, оскільки всі команди надсилаються одразу.

Підвищення продуктивності:

Підвищує продуктивність при виконанні великої кількості команд.

Ефективне використання мережі:

Зменшує кількість мережевих пакетів, що передаються між клієнтом і сервером.

Обмеження Pipelining

Пам'ять:

Використання Pipelining може збільшити використання пам'яті на клієнті та сервері, оскільки всі команди і відповіді зберігаються в пам'яті до завершення виконання.

Порядок виконання:

Всі команди в Pipelining виконуються в порядку їх надсилання, але якщо одна з команд зазнає невдачі, це не вплине на інші команди.

Резервне копіювання Redis

Резервне копіювання Redis — це процес створення копій даних, збережених у Redis, для забезпечення їхньої безпеки та відновлення у разі збою.

Redis підтримує кілька методів резервного копіювання, включаючи знімки (**snapshots**) і журнал транзакцій (**AOF**).

Знімки (Snapshots)

Знімки (RDB-файли) створюють повну копію даних у певний момент часу.

Переваги: Швидке створення резервних копій, менше використання дискового простору.

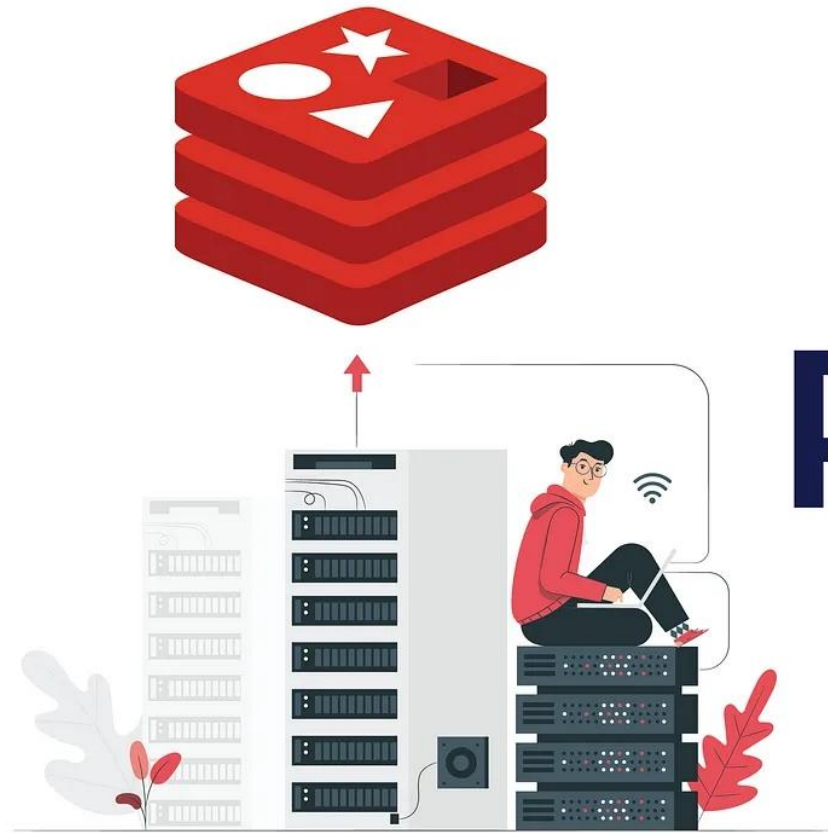
Недоліки: Можлива втрата даних, створених після останнього знімка.

Журнал транзакцій (Append-Only File, AOF)

AOF зберігає всі операції запису, виконані на сервері, у вигляді журналу.

Переваги: Менша ймовірність втрати даних, оскільки всі операції записуються.

Недоліки: Більше використання дискового простору, повільніше відновлення.



Redis

ДЯКУЮ ЗА УВАГУ!